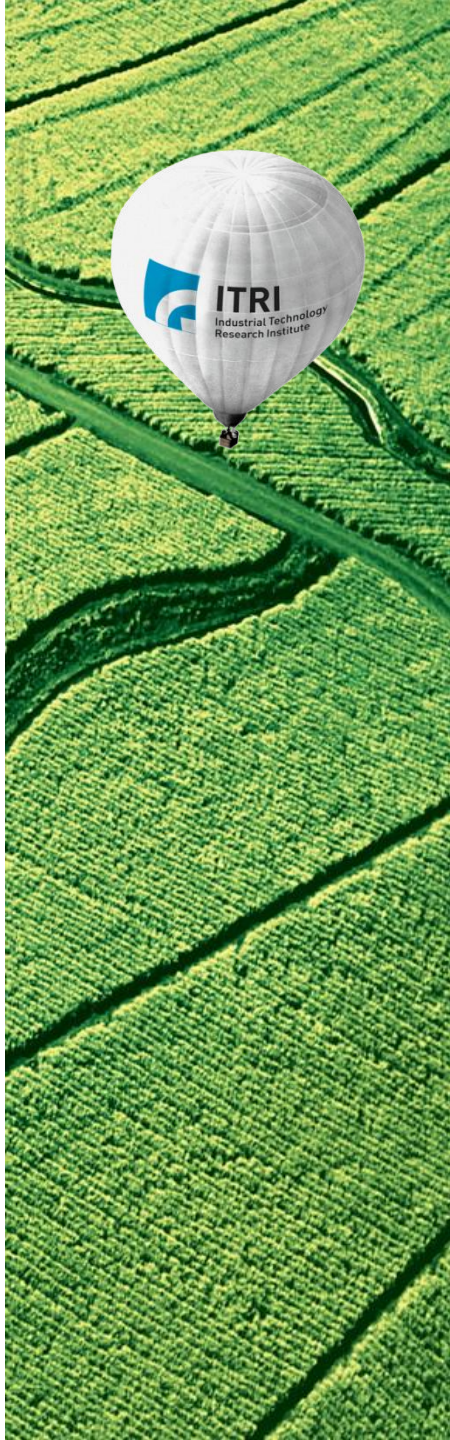
工業技術研究院

Industrial Technology
Research Institute

Collective-Capable AXI4 NoC Target Specification

Revision 0.2, 2026-07-23, draft



Overview

- **Collective-capable, AMBA AXI4-compliant NoC for transformer attention accelerators**
  - One compute tile per node on a 2D mesh, standard AXI4 at every endpoint
- **Highlights**
  - Hardware multicast on the write path, up to **15x** less source-port traffic on a 16-node set
  - Write-response aggregation: one AXI response per multicast write, error BRESP wins
  - Traffic-class isolation: bulk bursts never queue ahead of control messages
  - AXI4 endpoints: unicast unchanged, a collective write encodes operation and mask in AWUSER

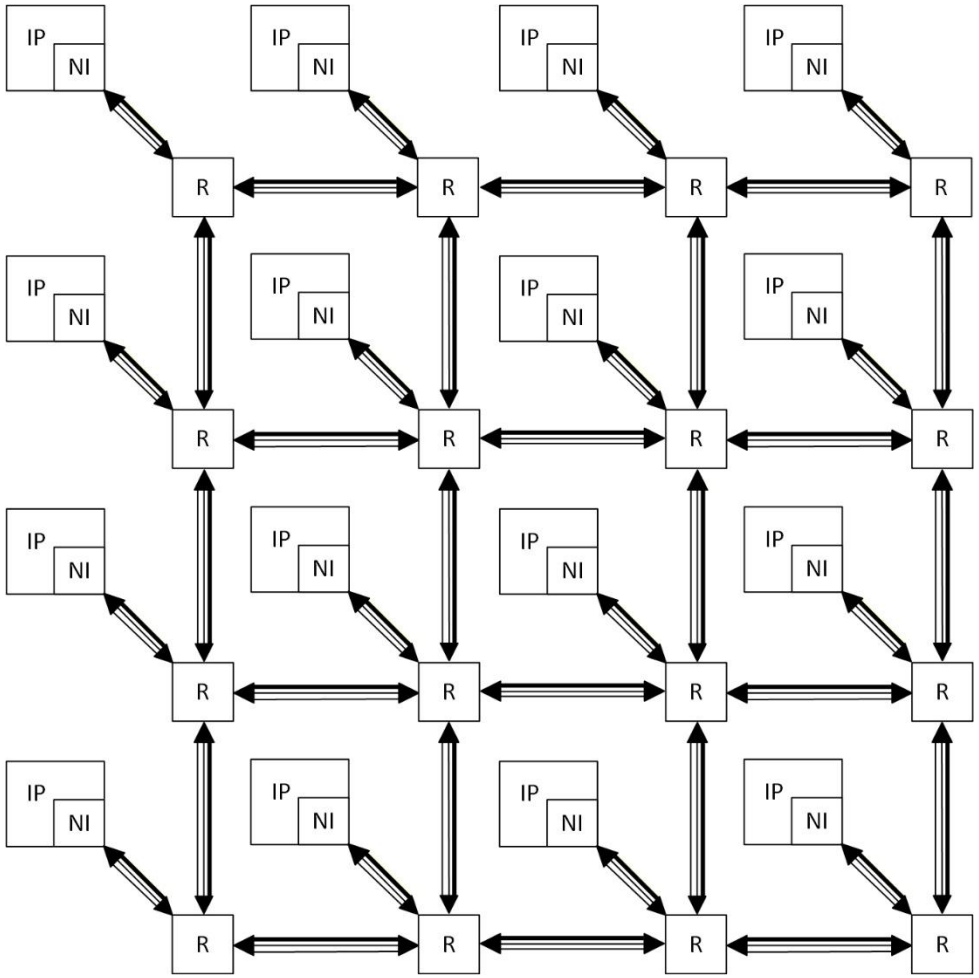
Class	Traffic	Properties	Network
High bandwidth	Q, K, V multicast, activations, KV cache blocks	Burst-based, up to 4 KB, latency tolerant	DAT, 512 b data plane, high utilization
Latency sensitive	Softmax statistics m and l, synchronization, scheduler start and done	Single-beat messages	Narrow 64 b control plane on REQ and RSP, low utilization
Auxiliary	Data-class requests DataAr, acknowledgments DataB, the merged multicast B	Small fixed-size flits	REQ and RSP

Key features and scope

- **Key features**
  - 1 GHz NoC clock target
  - Full AXI4, multiple outstanding transactions and same-ID reordering included
  - A multicast write injects once and returns one merged B, up to **15x** less source-port traffic
  - Physical REQ/RSP/DAT separation, a receiver accepts responses while its request path is blocked
  - 64 b control plus 512 b data class, **8 + 64 GB/s** per port at 1 GHz
  - XY wormhole fabric, deadlock-free at one VC per network channel
  - GALS integration, each endpoint keeps its own clock
- **Scope**
  - A multicast covers an aligned submesh, not an arbitrary node set. Software assigns it, the fabric provides the primitive
  - Write path only. Reads and read data are always unicast
  - No arithmetic on payload anywhere in the fabric. Tiles combine partial results, softmax statistics included, over ordinary unicast

Architecture

- Each node holds one compute tile and its local memory, so it is both an AXI master and an AXI slave
- XY routing fixes the path a multicast takes, its merged response retraces that path in reverse



- every link, per direction, flit wires only
- REQ 152 b + RSP 126 b + DAT 628 b
- 4 x 4 shown, 16 x 16 max

工業技術研究院
Industrial Technology
Research Institute

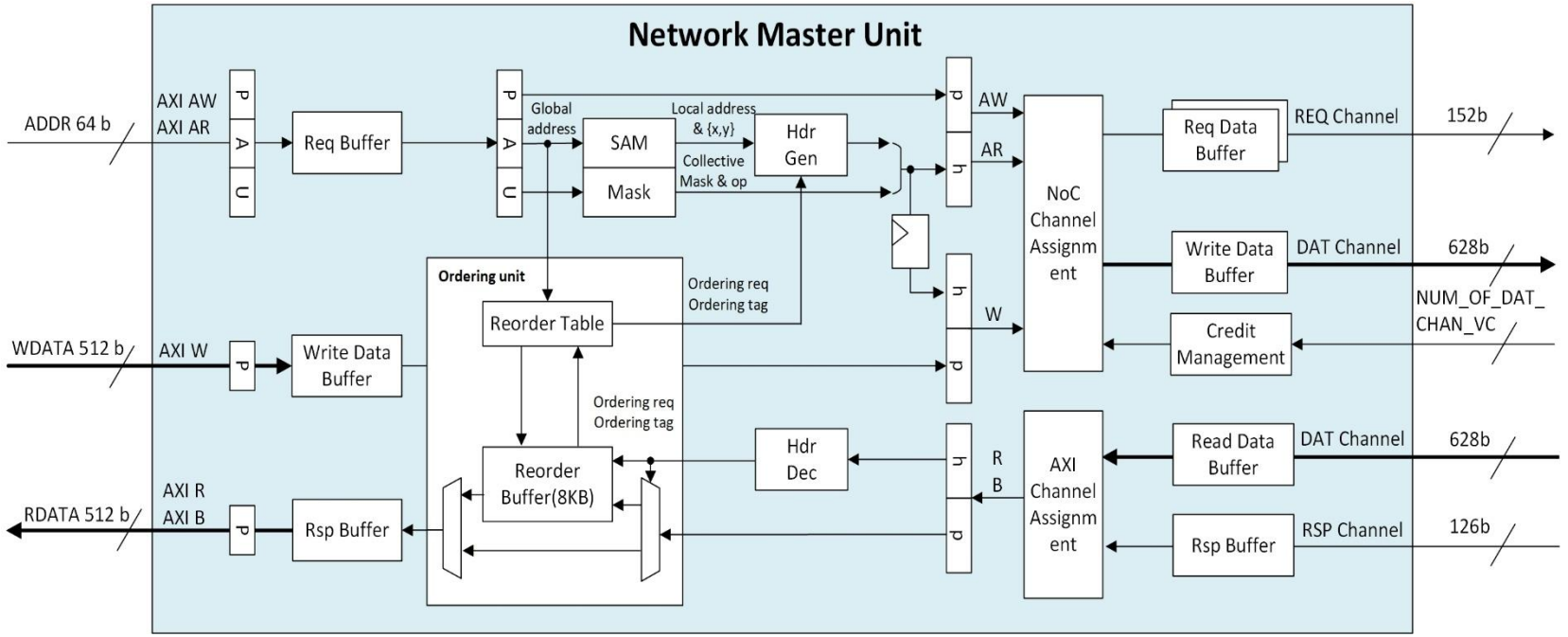
Physically separate networks

- The REQ/RSP split is a correctness requirement, a node must accept responses while its own request path is blocked
- A DAT endpoint sinks every delivered flit, write data into slave-side buffers, read data into reorder-buffer space reserved at request issue, so the two payload flows form no blocking cycle
- A split would double the widest wires of the design for no correctness gain

		Mapping & primary payload	
Physical link	**Size**	**Narrow class**	**Data class**
REQ	152b	Aw, Ar: 64 b address W: 64 b data	Ar: 64 b address
RSP	126b	R: 64 b data B: 2 b response	B: 2 b response
DAT	628b	-	Aw: 64 b address W, R: 512 b data

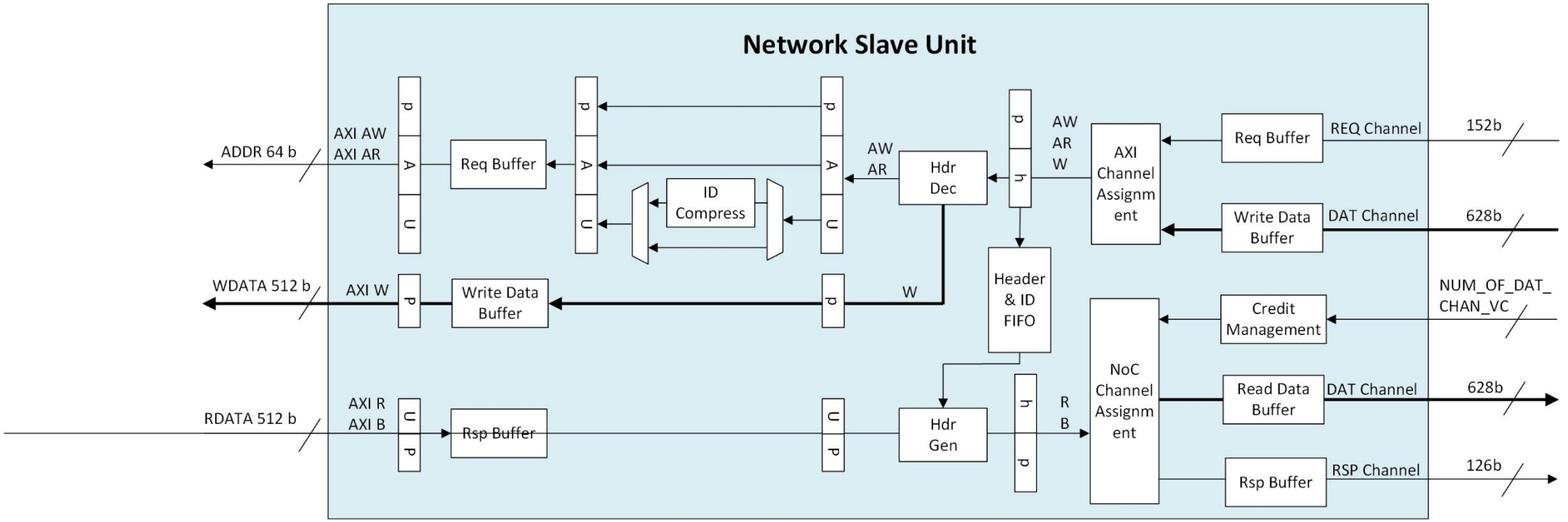
NMU (Network Master Unit)

- **Request path**
  - Round-robin arbitration with wormhole lock
  - SAM(System Address Map)
    - Global Address Translate to Local Address
  - Packetizer
- **Response path**
  - Depacketizer
  - Reorder buffer: 8 KB SRAM, two responses at the AXI 4 KB maximum burst
    - Same-ID ordering: same-destination streams bypass the reorder buffer



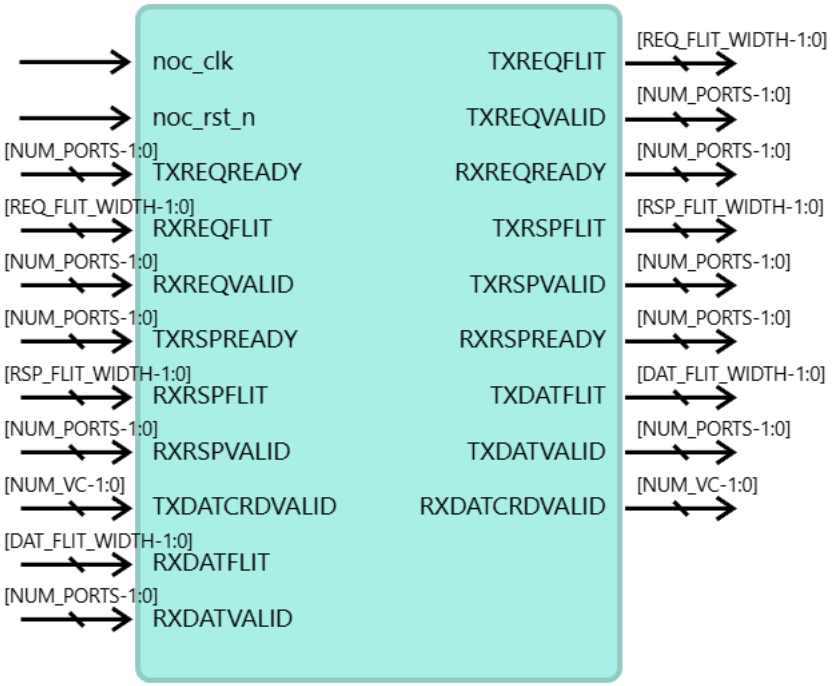
NSU (Network Slave Unit)

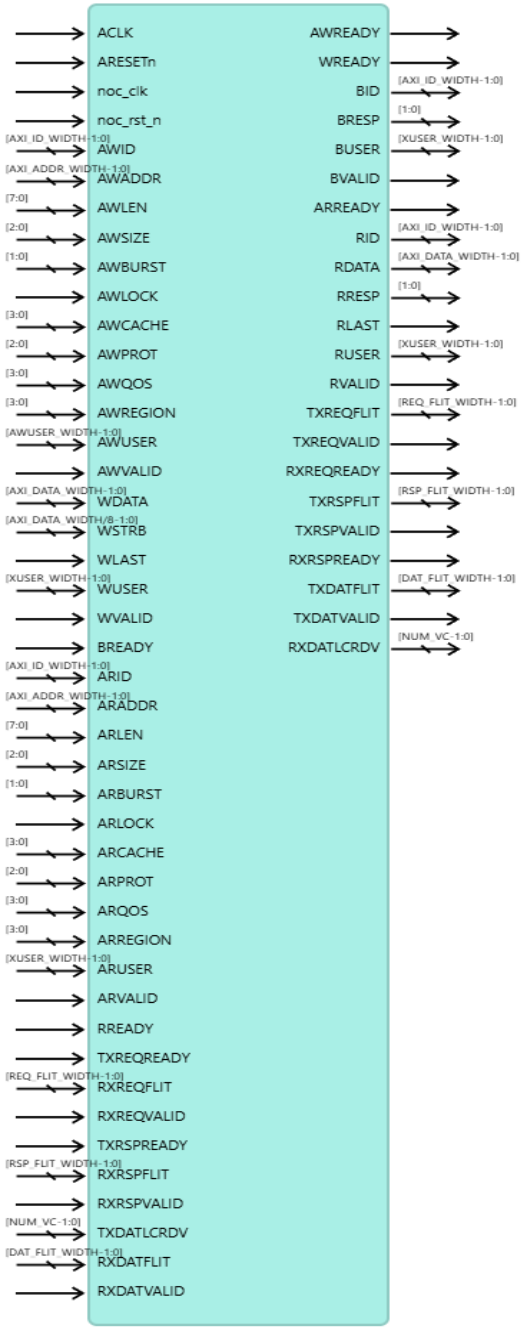
- **Request path**
  - Packet buffer
  - ID compression: narrower downstream ID width
    - Streams sharing a compressed ID return in order
  - Depacketizer to AW, AR, W
- **Response path**
  - Header FIFO: header kept for response packetization
  - B and R buffers, round-robin arbitration
  - Packetizer

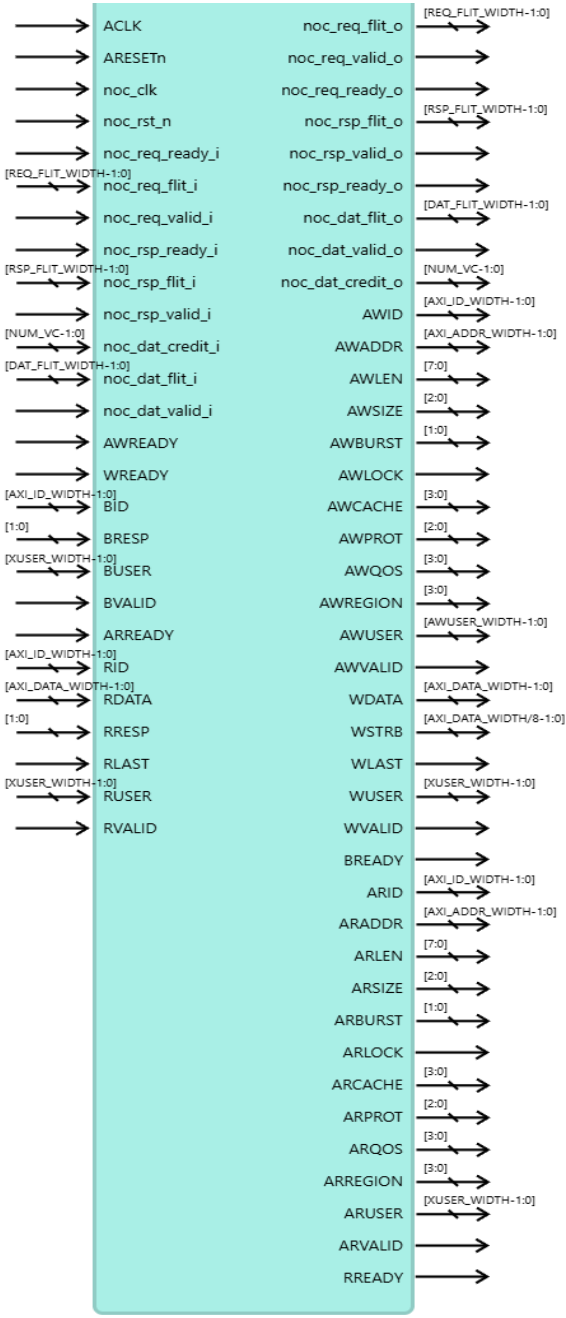


NI & Router Interfaces

Unit	AXI interface	NoC interface
NMU	AXI slave interface	NoC initiator interface
NSU	AXI master interface	NoC target interface
Router	-	NoC link interface per port, one local and four directional (N, E, S, W)







AXI channel signals

The NMU and NSU ports carry the same AXI4 channel set at the widths of section 5, so the table covers both.

Channel	Signals	Key widths
Write address AW	AWID, AWADDR, AWLEN, AWSIZE, AWBURST, AWLOCK, AWCACHE, AWPROT, AWQOS, AWREGION, AWUSER, AWVALID, AWREADY	AWADDR 64, AWID 8, AWUSER 18
Write data W	WDATA, WSTRB, WLAST, WUSER, WVALID, WREADY	WDATA 64 or 512, WSTRB 8 or 64
Write response B	BID, BRESP, BUSER, BVALID, BREADY	BID 8, BRESP 2, BUSER 8
Read address AR	ARID, ARADDR, ARLEN, ARSIZE, ARBURST, ARLOCK, ARCACHE, ARPROT, ARQOS, ARREGION, ARUSER, ARVALID, ARREADY	ARADDR 64, ARID 8, ARUSER 8
Read data R	RID, RDATA, RRESP, RLAST, RUSER, RVALID, RREADY	RDATA 64 or 512, RRESP 2

NoC link interface

- Each link carries the three physical networks, each an independent signal group per direction
  - REQ and RSP use ready/valid flow control
    - One Virtual Channel Implement
  - DAT uses credit-based flow control
    - Multiple Virtual Channel Implement

Network	Flit width	Flow control
REQ	152	valid and ready, one instance per direction
RSP	126	valid and ready, one instance per direction
DAT	628	valid plus per-virtual-channel credit return, no ready

Configuration options

Feature	Parameter	Values (default)	Comments
Topology	Mesh X and Y dimension	2-16 (4)	Square meshes only. 256 nodes maximum, set by the 8-bit node ID
AXI interface	Endpoint interfaces	1, 2 (1)	One interface carries both classes, with the class selected by the SAM address space. Two carry one class each
Flow control	NUM_VC, virtual channels per network channel	1-8 (1)	Sets the NoC link interface credit signal width
Ordering	Outstanding transactions per ID	1-32 (32)	1 when same-ID response reordering is disabled. A master holds at most 256 in total, one per ordering_tag, see section 6
Ordering	Same-ID response reordering	enabled, disabled (enabled)	The ordering requirement holds in either setting
Address map	SAM address spaces	config, memory	Config space selects the narrow class, memory space the data class. Uniform across nodes, loaded at runtime
Address map	Space region size	power of two	-

NoC Communication Protocol – Flit header

Field	Bits	Width	Description
axi_ch	[3:0]	4	AXI channel of the flit. 0: NarrowAw, on REQ 1: NarrowW, on REQ 2: NarrowAr, on REQ 3: NarrowB, on RSP 4: NarrowR, on RSP 5: DataAw, on DAT 6: DataW, on DAT 7: DataAr, on REQ 8: DataB, on RSP 9: DataR, on DAT
src_id	[11:4]	8	Source node id, composed as {y[3:0], x[3:0]}
dst_id	[19:12]	8	Destination node id, same composition
vc_id	[22:20]	3	Virtual channel index, 0 to the configured VC count minus one
flit_tail	[23]	1	Indicates the last flit of a packet
ordering_req	[24]	1	When asserted, ordering_tag is valid
ordering_tag	[32:25]	8	Reorder slot handle, allocated per transaction. Bounds a master to 256 outstanding transactions
collective_op	[34:33]	2	Collective operation. 0: UNICAST, single destination, collective_mask zero 1: MULTICAST, replicate a request to the node set of collective_mask, merge its responses, error BRESP first 2-3: reserved Write path only, Ar and R are always UNICAST
collective_mask	[42:35]	8	Names the multicast node set: wildcard over dst_id on a request, src_id on its response. Derived from the AWUSER address mask

NoC Communication Protocol – Flit header (cont.)

Field	Bits	Width	Description
collective_op	[34:33]	2	Collective operation. Write path only, Ar and R are always UNICAST 0: UNICAST 1: MULTICAST 2-3: reserved
collective_mask	[42:35]	8	Wildcard node mask, same composition as dst_id. A set bit marks that id bit as don't care, qualifying dst_id on a request and src_id on its response. Set bits are contiguous from the low end of each half, limited to the bits the mesh uses. The a low x and b low y bits name a 2^a by 2^b submesh aligned to its own size, origin dst_id with masked bits cleared. The set includes the source, which is not a destination of its own multicast

Flit payload, requests and write data

Aw, Ar, 109 b	Bits	Width
id	[7:0]	8
addr	[71:8]	64
len	[79:72]	8
size	[82:80]	3
burst	[84:83]	2
cache	[88:85]	4
lock	[89]	1
prot	[92:90]	3
region	[96:93]	4
qos	[100:97]	4
user	[108:101]	8

NarrowW, 81 b	Bits	Width
last	[0]	1
user	[8:1]	8
strb	[16:9]	8
data	[80:17]	64

DataW, 585 b	Bits	Width
last	[0]	1
user	[8:1]	8
strb	[72:9]	64
data	[584:73]	512

Flit payload, responses and AWUSER

B, 18 b	Bits	Width
id	[7:0]	8
resp	[9:8]	2
user	[17:10]	8

DataR, 531 b	Bits	Width
last	[0]	1
id	[8:1]	8
resp	[10:9]	2
user	[18:11]	8
data	[530:19]	512

NarrowR, 83 b	Bits	Width
last	[0]	1
id	[8:1]	8
resp	[10:9]	2
user	[18:11]	8
data	[82:19]	64

AWUSER, 18 b	Bits	Width
user	[7:0]	8
collective_op	[9:8]	2
collective_mask	[17:10]	8

ARUSER is 8 b.

Field utilization

Field utilization counts the 43 b header plus the flit payload over the flit width. Data utilization counts the AXI data field alone. Each flit width is the header plus its network's largest payload, so the largest channel fills its flit.

Network	Channel	Payload	Field	Data
REQ 152 b	NarrowAw, NarrowAr, DataAr	109 b	100.0 %	
REQ 152 b	NarrowW	81 b	81.6 %	42.1 %
RSP 126 b	NarrowR	83 b	100.0 %	50.8 %
RSP 126 b	NarrowB, DataB	18 b	48.4 %	
DAT 628 b	DataW	585 b	100.0 %	81.5 %
DAT 628 b	DataR	531 b	91.4 %	81.5 %
DAT 628 b	DataAw	109 b	24.2 %	

DataAw rides DAT for ordering, not size: AXI4 write data carries no ID, a slave pairs data with requests by arrival order, so the request travels bundled ahead of its data. The cost is one flit per write burst and amortizes over the burst length.

Metrics

Metric	Definition
Channel bandwidth	One flit per cycle per link direction, flit width x clock / 8
Payload bandwidth	AXI payload the same link carries, data width x clock / 8
Payload efficiency	Data beats over flits carried
Zero-load latency	Packet latency with no queueing and no contention
Sustained bandwidth	Payload bandwidth a channel delivers over a continuous stream

All figures assume AXI and NoC on one 1 GHz clock, the 512 b data class, and 4 KB bursts.

Payload bandwidth

A physical network carries one flit per cycle per direction and a flit holds at most one beat of AXI payload.

- **AXI interface, per port**
  - Narrow W, R: 64 b @ 1 GHz = **8 GB/s** each
  - Data W, R: 512 b @ 1 GHz = **64 GB/s** each
- **NoC channel, per link direction**
  - REQ: flit 152 b @ 1 GHz = 19.0 GB/s channel, 8 GB/s payload
  - RSP: flit 126 b @ 1 GHz = 15.8 GB/s channel, 8 GB/s payload
  - DAT: flit 628 b @ 1 GHz = 78.5 GB/s channel, **64 GB/s** payload
  - NarrowW rides REQ, NarrowR rides RSP: concurrent
  - Each DAT direction is its own wire bundle, a node's write data outbound and its read data inbound, both at 64 GB/s. Flows sharing one direction split it

Payload efficiency

The bandwidth problem with serialization

- Write
  - Shares DAT with AW+W
  - One flit per burst
  - N/(N+1) payload efficiency
- Read
  - Response-direction DAT to itself
  - No flit shared
  - 100% payload efficiency

Burst length	Payload efficiency
1 beat (64 B)	50%
4 beats	80%
16 beats	94.1%
64 beats (4 KB)	98.5%

Zero-load latency

- A packet crossing H hops traverses H links and passes H + 1 routers, counting the source router.
  - REQ / RSP: simple-mode router, ready/valid, **1 to 2** pipeline stages per hop
  - DAT: standard-mode router, multi-VC credit-based flow control, **3 to 5** pipeline stages per hop

$$T_{\text{zero\_load}} = H\, t_{\text{wire}} + (H+1)\, t_{\text{router}} + L$$

H	Manhattan hop count from source to destination
t_{wire}	link traversal delay per hop, in NoC cycles
t_{router}	router pipeline latency per hop, in NoC cycles
L	payload length in flits, one injected per cycle, so serialization contributes L cycles

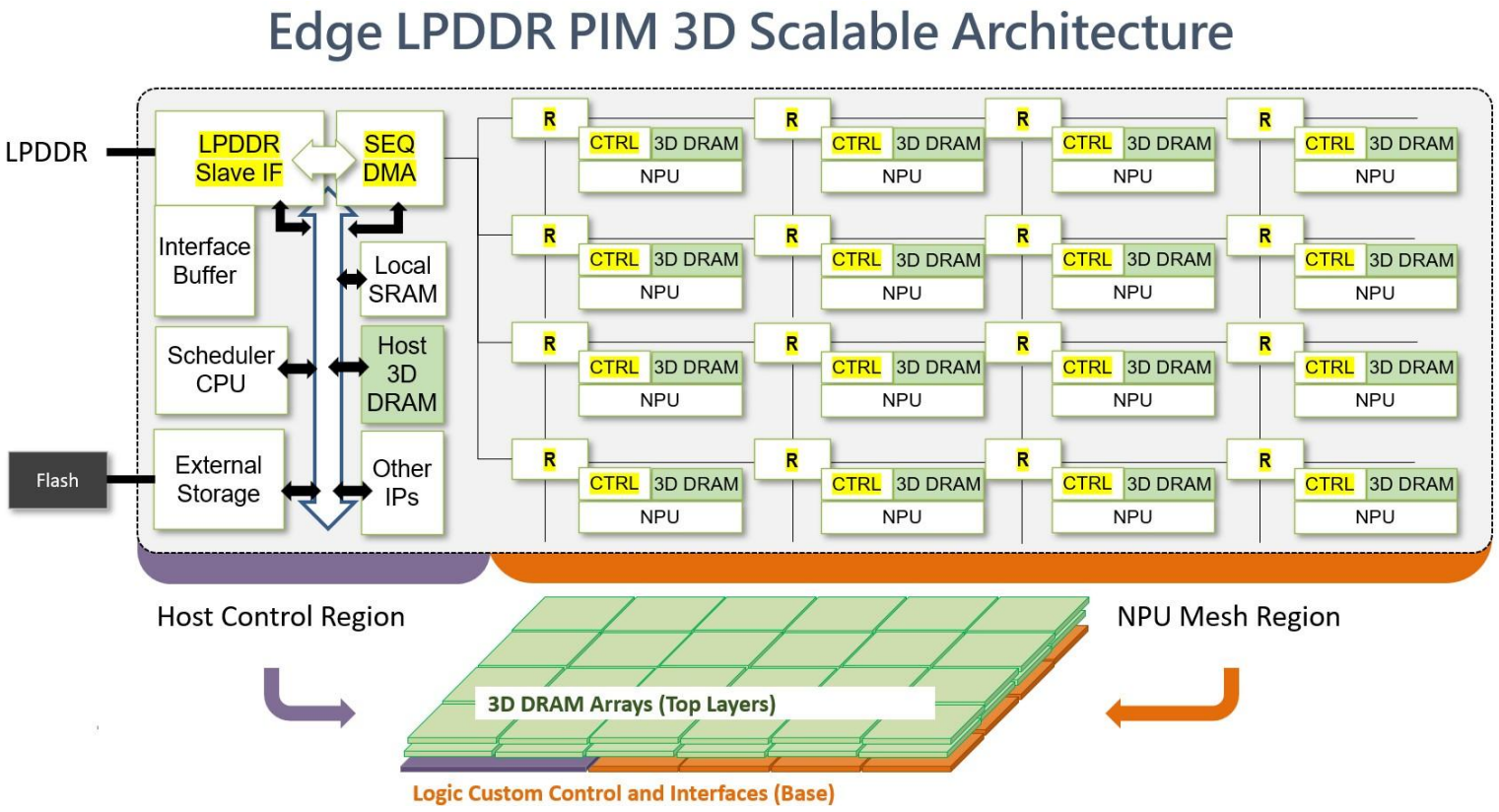
Sustained bandwidth

- In-order reads hold the full rate, 64 GB/s.
- Out-of-order reads lift efficiency across destinations, but the reorder-buffer size is their bottleneck
  - every reordered response passes through it, and once its 8 KB is fully allocated the next request waits.

Burst size	Request cap	Buffer cap (8KB/Burst_size)	Outstanding	Pending data (B)	Coverage (cycles) (pending data / 64 B per cycle)
64B	32	128	**32**	2048	32
256B	32	32	**32**	8192	128
1024B	32	8	**8**	8192	128
4096B	32	2	**2**	8192	128

Evaluation Model

The evaluation platform is the Edge LPDDR PIM 3D scalable architecture. The NoC under test is the router mesh and network interfaces of its NPU mesh region.



Edge LPDDR PIM 3D scalable architecture

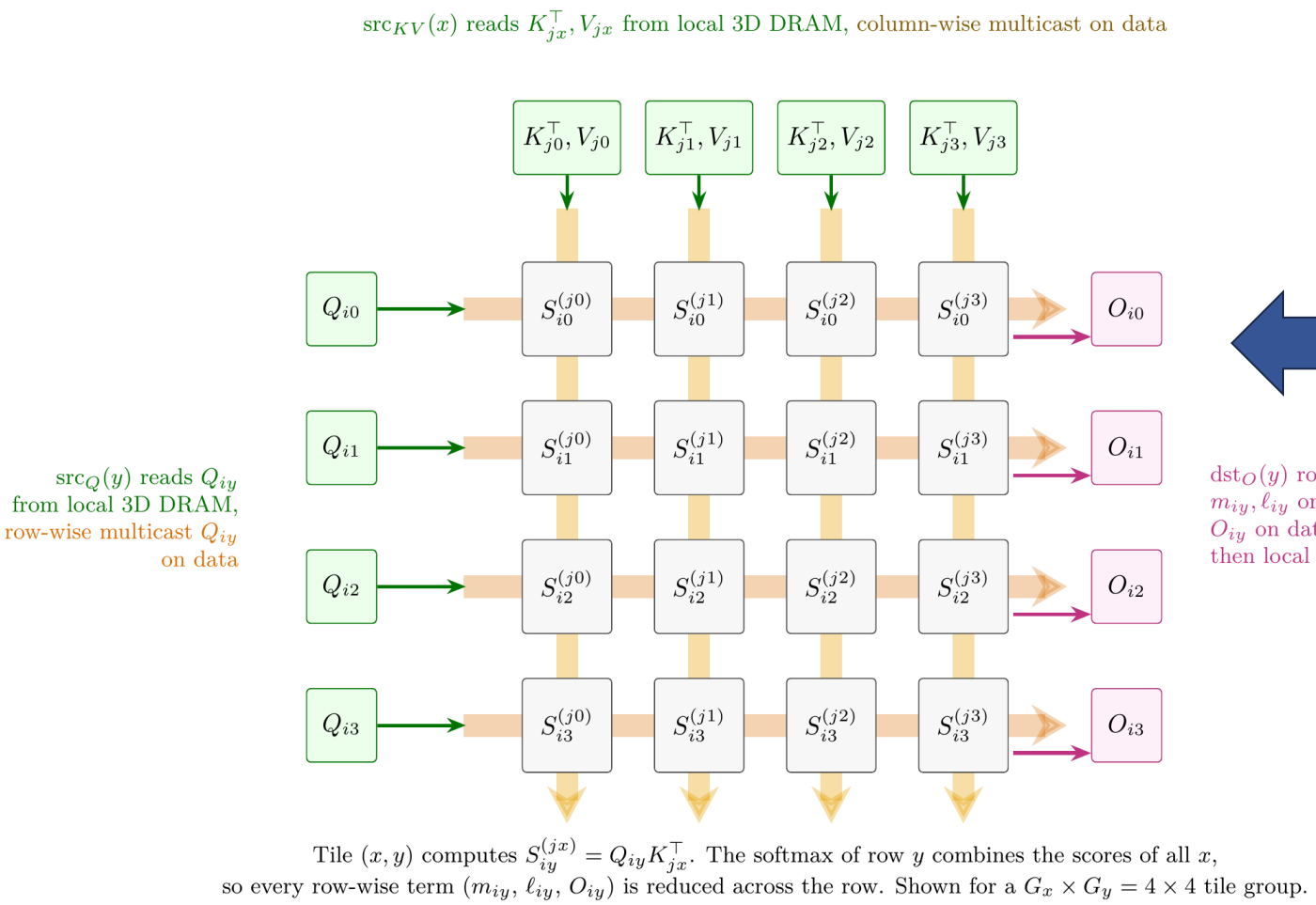
- Host control region: a scheduler CPU and a sequencer DMA stage data from LPDDR and external storage into the mesh
- NPU mesh region: one tile per node. A tile holds an NPU, its controller, tile-local 3D DRAM, and a router
- The 3D DRAM arrays occupy the top layers. The control and interface logic sits at the base

Memory tiers, cold storage to compute

Tier	Location	Role
External storage	Host control region, loaded from Flash	Persistent model store, read once at model load
LPDDR	External DRAM behind the LPDDR interface	Capacity tier. Holds the model image and feeds weight staging
Host 3D DRAM	Host control region	Bandwidth tier on the host side. KV cache offload target and sink of mesh output
Interface buffer	Host control region	Staging buffer between the external interfaces and the sequencer DMA
Local SRAM	Host control region	Scheduler working memory, holds DMA descriptors
Tile 3D DRAM	Every mesh tile	Near-memory tier. Resident weights, the tile's Q, K, V, O blocks, and hot KV cache. Tile-local, so a source-tile load generates no NoC traffic
L1	Inside the NPU	Compute scratchpad. Sub-tiles, partial sums, and online-softmax state, filled by tiling with double buffering

Tiled attention dataflow

Three algorithms specify the dataflow: model-level execution flow at the run level, and the attention block it invokes as per-tile fetch or multicast , differing only in how shared operands reach the tile group.



Attention block mapping onto a 4 x 4 tile group

Attention block mapping

A tile group computes one attention block : the score grid S = Q K^T maps onto the mesh, rows share Q, columns share K/V

工業技術研究院
Industrial Technology Research Institute

Model-level execution flow

Model-level execution flow : Weights load once into per-tile 3D DRAM and stay resident, no layer fetches weights over the fabric. A layer puts two traffic classes on the fabric:

- Feature pipeline: activations passed to the next layer, on the data plane.
- Scheduler control: per-group start and done messages, on the narrow control plane.

Layers map to tile groups round-robin as pipeline stages. A group holds the weights and KV cache of its layers, so only the activations X cross group boundaries:

Algorithm 0: Model-level execution flow on the tile array

Require: Weights W_Q, W_K, W_V, W_O and the token stream in LPDDR; weights stay resident in 3D DRAM once loaded. Tile groups and their source/owner roles as in Algorithms 1–2.

Load W_Q, W_K, W_V, W_O from LPDDR into per-tile 3D DRAM by host DMA. `// step 1: weight → 3D DRAM`

Bind the static schedule: assign $\mathrm{src}_Q(y)$, $\mathrm{src}_{KV}(x)$, $\mathrm{dst}_O(y)$ for every tile group. `// step 2: schedule → static flow`

Compute $X = \mathrm{embed}(\mathrm{tokens})$.

for $1 \le l \le N$ (layers) **do**

  Each tile group: compute $Q = XW_Q$, $K = XW_K$, $V = XW_V$, write the slices to per-tile 3D DRAM.

  Append K, V to the KV cache in tile 3D DRAM, evict cold blocks, reload on demand. `// step 4: KV ⇄ host 3D DRAM`

  Run Algorithm 2 on each tile group, compute O. `// step 2 executes data+narrow`

  Scheduler marks each group done. `// narrow`

  Compute $X = \mathrm{FFN}/\mathrm{residual}$ over O, pass it to the next layer. `// step 3: feature pipeline data`

end

Write the last-layer X to host 3D DRAM for the scheduler to sample the next token. `// results → host 3D DRAM data`

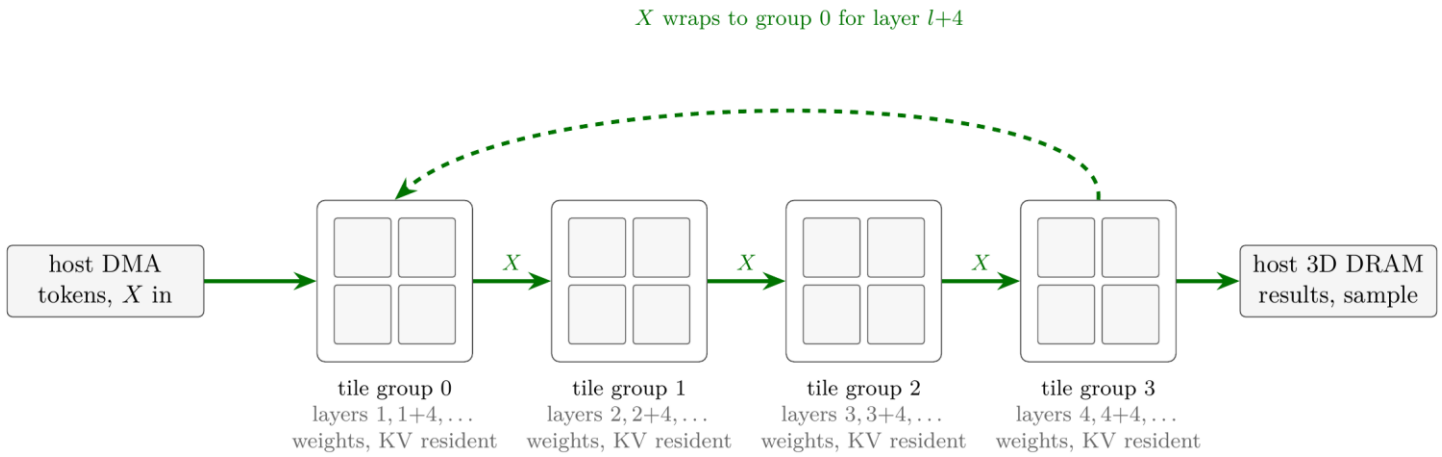
Return tokens = detokenize(X).

Algorithms 1 and 2

Algorithms 1 and 2 differ in one place, how a shared slice reaches its consumers:

- Algorithm 1 (per-tile fetch): every consumer tile pulls the slice itself, Gx unicast reads per Q slice and Gy per K/V slice (lines 5 and 7)
- Algorithm 2 (multicast): the source tile reads its slice locally once and multicasts it, one transfer per slice (lines 5-6 and 8-9)

In both figures the comments mark plane use: data for the Q, K, V, O tensors, narrow for the m and l statistics, local read/write for 3D DRAM accesses that generate no NoC traffic.



Feature pipeline: the layer l output X moves to the next group on the data network.
Each group runs the attention block mapping internally (Algorithm 2), then FFN/residual.
Weights never move after staging, so only X crosses group boundaries. Shown for a 4×4 mesh as four 2×2 groups.

Algorithm 1
Per-tile fetch

Algorithm 1 Per-tile fetch

Every consumer tile pulls the slice itself, Gx unicast reads per Q slice and Gy per K/V slice.

Algorithm 1: Per-tile fetch (unicast baseline) on a $G_x \times G_y$ tile group

Require: $Q, K^\top, V, O \in \mathbb{R}^{S\times D}$ resident in per-tile 3D DRAM, placed from LPDDR by host DMA before this block. Block sizes B_c, B_r, tile group sizes G_x, G_y. In the following algorithm, x, y denote tile coordinates relative to the group.

Divide Q, O into $T_r = \lceil S/B_r \rceil$ blocks $Q_1, \ldots, Q_{T_r} \in \mathbb{R}^{B_r\times D}$ and $O_1, \ldots, O_{T_r}$. Divide $K^\top, V$ into $T_c = \lceil S/B_c \rceil$ blocks.

Further divide each Q_i, O_i into G_y slices $Q_{i1}, \ldots, Q_{iG_y} \in \mathbb{R}^{(B_r/G_y)\times D}$, and each $K_j^\top, V_j$ into G_x slices. Slices are accessed according to tile coordinates x, y.

for $1 \le i \le T_r$ **do**

  On chip, initialize $O_{iy}^{(0x)} = 0$.

  each tile (x, y): read Q_{iy} from $\mathrm{src}_Q(y)$'s 3D DRAM to on-chip L1. `// G_x unicast reads per slice data`

  for $1 \le j \le T_c$ **do**

    each tile (x, y): read $K_{jx}^\top, V_{jx}$ from $\mathrm{src}_{KV}(x)$'s 3D DRAM to on-chip L1. `// G_y unicast reads per slice data`

    Tile on-chip compute $S_{iy}^{(jx)} = Q_{iy} K_{jx}^\top$.

    Tile on-chip compute $S_{iy}^{(jx)} = S_{iy}^{(jx)}/\sqrt{D}$, $m_{iy}^{(jx)} = \mathrm{rowmax}(S_{iy}^{(jx)})$.

    if $j > 1$ **then**

      Tile on-chip compute $m_{iy}^{(jx)} = \max(m_{iy}^{(j-1)}, m_{iy}^{(jx)})$.

    end

    (row owner $\mathrm{dst}_O(y)$) Row-wise reduce $m_{iy}^{(j)} = \max(m_{iy}^{(jx)})$ in tile group. `// narrow`

    (row owner $\mathrm{dst}_O(y)$) Row-wise multicast $m_{iy}^{(j)}$ in tile group. `// narrow`

    Tile on-chip compute $\tilde{P}_{iy}^{(jx)} = \exp(S_{iy}^{(jx)} - m_{iy}^{(j)})$.

    Tile on-chip compute $\ell_{iy}^{(jx)} = \mathrm{rowsum}(\tilde{P}_{iy}^{(jx)})$.

    (row owner $\mathrm{dst}_O(y)$) Row-wise reduce $\ell_{iy}^{(j)} = \sum(\ell_{iy}^{(jx)})$ in tile group. `// narrow`

    (row owner $\mathrm{dst}_O(y)$) Row-wise multicast $\ell_{iy}^{(j)}$ in tile group. `// narrow`

    if $j > 1$ **then**

      Tile on-chip compute $\ell_{iy}^{(j)} = e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}} \ell_{iy}^{(j-1)} + \ell_{iy}^{(j)}$.

      Tile on-chip compute $O_{iy}^{(jx)} = \mathrm{diag}\big(e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}}\big) O_{iy}^{(j-1)x}$.

    end

    Tile on-chip compute $O_{iy}^{(jx)} = O_{iy}^{(jx)} + \tilde{P}_{iy}^{(jx)} V_{jx}$.

    Tile on-chip update $m_{iy}^{(j-1)} \leftarrow m_{iy}^{(j)}$, $\ell_{iy}^{(j-1)} \leftarrow \ell_{iy}^{(j)}$.

  end

  Tile on-chip compute $O_{iy}^{(T_c)x} = \mathrm{diag}(\ell_{iy}^{(T_c)})^{-1} O_{iy}^{(T_c)x}$.

  (row owner $\mathrm{dst}_O(y)$) Row-wise reduce $O_{iy}^{(T_c)} = \sum(O_{iy}^{(T_c)x})$ in tile group. `// data`

  (row owner $\mathrm{dst}_O(y)$) Write O_{iy} to its local 3D DRAM as the y-th slice in i-th block of O. `// local write`

end

Algorithm 2 Multicast

Algorithm 2 Multicast. The source tile reads its slice locally once and multicasts it, one transfer per slice.

Algorithm 2: Multicast on a $G_x \times G_y$ tile group (FlatAttention structure on per-tile 3D DRAM)

Require: $Q, K^\top, V, O \in \mathbb{R}^{S\times D}$ resident in per-tile 3D DRAM, placed from LPDDR by host DMA before this block. Block sizes B_c, B_r, tile group sizes G_x, G_y. In the following algorithm, x, y denote tile coordinates relative to the group.

Divide Q, O into $T_r = \lceil S/B_r \rceil$ blocks $Q_1, \ldots, Q_{T_r} \in \mathbb{R}^{B_r\times D}$ and $O_1, \ldots, O_{T_r}$. Divide $K^\top, V$ into $T_c = \lceil S/B_c \rceil$ blocks.

Further divide each Q_i, O_i into G_y slices $Q_{i1}, \ldots, Q_{iG_y} \in \mathbb{R}^{(B_r/G_y)\times D}$, and each $K_j^\top, V_j$ into G_x slices. Slices are accessed according to tile coordinates x, y.

for $1 \le i \le T_r$ **do**

  On chip, initialize $O_{iy}^{(0x)} = 0$.

  (row source $\mathrm{src}_Q(y)$) Load Q_{iy} from local 3D DRAM to tile on-chip L1. `// local read`

  (row source $\mathrm{src}_Q(y)$) Row-wise multicast Q_{iy} in tile group. `// data`

  for $1 \le j \le T_c$ **do**

    (column source $\mathrm{src}_{KV}(x)$) Load $K_{jx}^\top, V_{jx}$ from local 3D DRAM to tile on-chip L1. `// local read`

    (column source $\mathrm{src}_{KV}(x)$) Column-wise multicast $K_{jx}^\top, V_{jx}$ in tile group. `// data`

    Tile on-chip compute $S_{iy}^{(jx)} = Q_{iy} K_{jx}^\top$.

    Tile on-chip compute $S_{iy}^{(jx)} = S_{iy}^{(jx)}/\sqrt{D}$, $m_{iy}^{(jx)} = \mathrm{rowmax}(S_{iy}^{(jx)})$.

    if $j > 1$ **then**

      Tile on-chip compute $m_{iy}^{(jx)} = \max(m_{iy}^{(j-1)}, m_{iy}^{(jx)})$.

    end

    (row owner $\mathrm{dst}_O(y)$) Row-wise reduce $m_{iy}^{(j)} = \max(m_{iy}^{(jx)})$ in tile group. `// narrow`

    (row owner $\mathrm{dst}_O(y)$) Row-wise multicast $m_{iy}^{(j)}$ in tile group. `// narrow`

    Tile on-chip compute $\tilde{P}_{iy}^{(jx)} = \exp(S_{iy}^{(jx)} - m_{iy}^{(j)})$.

    Tile on-chip compute $\ell_{iy}^{(jx)} = \mathrm{rowsum}(\tilde{P}_{iy}^{(jx)})$.

    (row owner $\mathrm{dst}_O(y)$) Row-wise reduce $\ell_{iy}^{(j)} = \sum(\ell_{iy}^{(jx)})$ in tile group. `// narrow`

    (row owner $\mathrm{dst}_O(y)$) Row-wise multicast $\ell_{iy}^{(j)}$ in tile group. `// narrow`

    if $j > 1$ **then**

      Tile on-chip compute $\ell_{iy}^{(j)} = e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}} \ell_{iy}^{(j-1)} + \ell_{iy}^{(j)}$.

      Tile on-chip compute $O_{iy}^{(jx)} = \mathrm{diag}\big(e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}}\big) O_{iy}^{(j-1)x}$.

    end

    Tile on-chip compute $O_{iy}^{(jx)} = O_{iy}^{(jx)} + \tilde{P}_{iy}^{(jx)} V_{jx}$.

    Tile on-chip update $m_{iy}^{(j-1)} \leftarrow m_{iy}^{(j)}$, $\ell_{iy}^{(j-1)} \leftarrow \ell_{iy}^{(j)}$.

  end

  Tile on-chip compute $O_{iy}^{(T_c)x} = \mathrm{diag}(\ell_{iy}^{(T_c)})^{-1} O_{iy}^{(T_c)x}$.

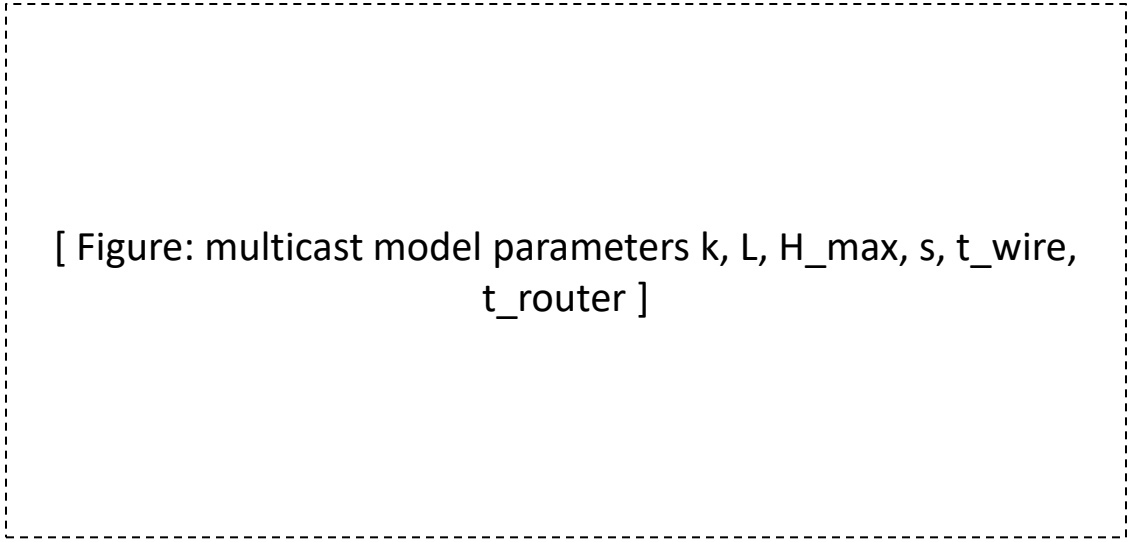
  (row owner $\mathrm{dst}_O(y)$) Row-wise reduce $O_{iy}^{(T_c)} = \sum(O_{iy}^{(T_c)x})$ in tile group. `// data`

  (row owner $\mathrm{dst}_O(y)$) Write O_{iy} to its local 3D DRAM as the y-th slice in i-th block of O. `// local write`

end

Multicast model and tail latency

The row and column multicasts of Algorithm 2 follow the model below, a multicast along one axis over a set of k nodes. A submesh multicast runs in two phases, one per axis.



Case	Tail arrival at the farthest member
Repeated unicast	(k - 1)L + H_max t_wire + (H_max + 1) t_router
In-network multicast	L + H_max t_wire + (H_max + 1) t_router

Multicast traffic

Traffic generated by one multicast, source at one end of the set:

Quantity	Unicast	In-network	Ratio, k = 4	Ratio, k = 16
Source port flits	(k - 1)L	L per path	3x	15x
Flit hops	L [s(s + 1)/2 + (k - 1 - s)(k - s)/2]	L(k - 1)	2x	8x

The transport term of latency is identical in both cases. Multicast replaces the (k - 1)L serialization at one source port with a single sub-tile, so both the tail latency and the source traffic fall with the set size. A source at one end of the set uses one path with H_max = k - 1. An interior source uses one path per direction, so H_max = floor(k/2), the port ratio halves, and the flit-hop ratio drops to 1.33x at k = 4 and 4.27x at k = 16.